Manual de Instalación

(Infraestructura, bases de datos, microservicios, aplicación y medidor)

Cuida Tu Agua: Plataforma web y móvil de monitoreo del consumo de agua con control remoto de la válvula

Juan Esteban Ome Esquivel

Juan Manuel Gómez

Diego Alejandro Alvarado

Equipo de desarrollo

Servicio Nacional de Aprendizaje — SENA

Centro de formación: Centro de la Industria, la Empresa y los Servicios

Regional: Regional Huila

Programa: Tecnología en Análisis y Desarrollo de Software

Ficha: 3145556

Instructor: Carlos Julio Cadena

9 de octubre de 2026

Versión: 1.0

Control de versiones

Tabla 1

Historial de versiones del documento

Versión	Fecha	Autor(es)	Descripción del cambio
1.0	09/10/2026	Equipo Cuida Tu Agua	Primera versión. Describe la instalación de la infraestructura, las bases de datos, los seis microservicios, la aplicación móvil y web, y el firmware del medidor.

Tabla 2

Revisión y aprobación del documento

Rol	Nombre	Firma	Fecha
Elaboró	Equipo de desarrollo		
Revisó	Carlos Julio Cadena (instructor)		
Aprobó			

Tabla de contenido

Control de versiones.....

Tabla de contenido.....

1. Introducción.....

1.1 Orden general de la instalación.....

2. Requisitos.....

2.1 Equipo servidor.....

2.2 Software.....

2.3 Hardware del medidor.....

3. Repositorios y carpetas.....

4. Base de datos.....

5. Infraestructura de apoyo.....

5.1 Redis y Mailpit.....

5.2 Mosquitto y RabbitMQ.....

6. Llaves y clave interna.....

6.1 Llaves RS256 de los tokens.....

6.2 Clave interna entre servicios.....

7. Configurar y arrancar ms-iam.....

8. Configurar y arrancar los servicios .NET.....

9. Firewall.....

10. Asignar el rol de administrador.....

11. Aplicación móvil y web.....

11.1 Configuración.....

11.2 Aplicación móvil.....

11.3 Aplicación web y panel de administración.....

12. Medidores.....

12.1 Alta de fábrica.....

12.2 Cargar el firmware.....

12.3 Prueba en banco.....

13. Verificación de la instalación.....

14. Solución de problemas.....

15. Archivos que nunca se suben a Git.....

Referencias.....

Para actualizar la tabla: clic derecho sobre ella → Actualizar campos → Actualizar toda la tabla.

1. Introducción

Este manual explica, paso a paso, cómo instalar Cuida Tu Agua en un equipo servidor: la infraestructura (SQL Server, Redis, Mosquitto, RabbitMQ y el servidor de correo de pruebas), las bases de datos, los seis microservicios, la aplicación móvil y web, y el firmware del medidor. Está dirigido a la persona del equipo que instala o reinstala el sistema.

Los comandos están escritos para Windows con PowerShell, que es el entorno del equipo de desarrollo; en Linux los comandos de Docker, .NET, Maven y npm son los mismos. Las contraseñas y claves nunca se escriben en este documento: donde aparece un valor entre ángulos, por ejemplo <CONTRASEÑA_SA>, se debe reemplazar por el valor real, que se guarda fuera de los repositorios.

1.1 Orden general de la instalación

1. Instalar los programas requeridos (sección 2).
2. Descargar los repositorios en la estructura de carpetas (sección 3).
3. Crear la base de datos y aplicar las migraciones (sección 4).
4. Levantar Redis, Mailpit, Mosquitto y RabbitMQ (sección 5).
5. Generar las llaves JWT y la clave interna (sección 6).
6. Configurar y arrancar los microservicios (secciones 7 y 8).
7. Abrir los puertos del firewall y asignar el rol de administrador (secciones 9 y 10).
8. Instalar la aplicación móvil y publicar la web (sección 11).
9. Preparar e instalar los medidores (sección 12).
10. Verificar la instalación (sección 13).

2. Requisitos

2.1 Equipo servidor

- Windows 10 u 11, o Linux, con Docker Desktop (o Docker Engine) funcionando.
- Al menos 2 GB de memoria libres para el contenedor de SQL Server, que es el requisito mínimo de su imagen.

- Conexión a la misma red WiFi de 2,4 GHz que los medidores y los celulares de prueba, con comunicación permitida entre dispositivos.

2.2 Software

Tabla 3

Software requerido

Programa	Versión	Para qué	Cómo comprobarlo
Docker Desktop	Reciente	SQL Server, Liquibase, Redis, Mailpit, Mosquitto y RabbitMQ	docker --version
JDK	21	ms-iam (Spring Boot 4.1.1, Maven Wrapper incluido)	java -version
.NET SDK	10.0.100 o superior (global.json)	ms-places, ms-device, ms-consumption, ms-valve, ms-notification	dotnet --version
Node.js y npm	Node 20 o superior; npm 10 o superior	Aplicación móvil y web (Expo SDK 57)	node -v; npm -v
OpenSSL	Incluido en Git para Windows	Generar las llaves RS256	openssl version
PowerShell 7	7 o superior	Script de alta de medidores por consola	pwsh --version
Thonny o mpremote	Reciente	Cargar el firmware en el ESP32	—
MicroPython para ESP32	1.22 o superior, con umqtt.simple	Ejecutar el firmware del medidor	import os; os.uname().release
Cliente HTTP (opcional)	REST Client de VS Code o IntelliJ	Probar las API con los archivos .http	—

2.3 Hardware del medidor

- ESP32 DevKit (WROOM) y cable USB de datos.
- Sensor de caudal YF-S201: rojo a 3V3, negro a GND y amarillo a GPIO4.
- Módulo de relé conectado a GPIO5 (GPIO26 si el relé hace clic al reiniciar) y electroválvula normalmente cerrada.
- Fuente de alimentación para el ESP32 y la electroválvula.

3. Repositorios y carpetas

Se recomienda la siguiente estructura en el equipo servidor; los comandos de este manual asumen esas rutas.

```
cuida-tu-agua\
├── sy-water-db\              repositorios de base de datos (Liquibase)
│   ├── ms-iam-db              schema security (crea la base sy-water-db)
│   ├── ms-places-db           schema places
│   ├── ms-device-db           schema devices
│   ├── ms-consumption-db      schema monitoring
│   ├── ms-valve-db            schema valve
│   └── ms-notification-db     schema notification
├── sy-water-backend\
│   ├── ms-iam                 Java 21 / Spring Boot
│   ├── ms-places              .NET 10
│   ├── ms-device              .NET 10 (incluye compose de Mosquitto y
RabbitMQ)
│   ├── ms-consumption         .NET 10
│   ├── ms-valve               .NET 10
│   └── ms-notification        .NET 10
├── sy-water-frontend\
│   └── save-your-water        aplicación Expo: móvil, web y panel de
administración
└── sy-water-hardware\
    └── firmware               MicroPython del ESP32
```

Tabla 4

Puertos del sistema

Componente	Puerto	Notas
SQL Server (sy_water_db_dev)	1433	Base de datos sy-water-db
ms-iam	3001	Salud: /actuator/health
ms-places	3002	Salud: /health
ms-device	3003	Salud: /health (incluye MQTT)
ms-consumption	3004	Salud: /health
ms-valve	3005	Salud: /health
ms-notification	3006	Salud: /health
Mosquitto (MQTT)	1883	Sin TLS; en producción debe usarse TLS en 8883
RabbitMQ	5672 / 15672	AMQP / panel web (usuario sywater)
Redis	6379	Lista de tokens revocados
Mailpit	1025 / 8025	SMTP de pruebas / bandeja web
Expo web (desarrollo)	8081	npx expo start --web
Web publicada	8080	Sugerido para nginx

4. Base de datos

Cada microservicio tiene su repositorio de base de datos con Liquibase. Todos siguen la misma estructura: sqlserver-init crea el schema y los usuarios, y Liquibase aplica los changesets de 01-ddl, 02-dml, 03-dcl, 04-tcl y 06-releases. Los repositorios se aplican en orden, porque cada schema tiene llaves foráneas hacia los anteriores.

Tabla 5

Orden de aplicación de las bases de datos

Orden	Repositorio	Schema	Usuario de la aplicación	Etiqueta final
1	ms-iam-db	security	security_app	v1.3-iam-action-codes
2	ms-places-db	places	places_app	Última release disponible
3	ms-device-db	devices	devices_app	v1.1-admin-registration
4	ms-consumption-db	monitoring	consumption_app	Última release disponible
5	ms-valve-db	valve	valve_app	Última release disponible
6	ms-notification-db	notification	notification_app	v1.4-device-liveness

En cada repositorio, en ese orden:

```
cd C:\...\cuida-tu-agua\sy-water-db\ms-iam-db
Copy-Item .env.example .env        # escribir las contraseñas en .env
(nunca a git)
docker compose up                  # crea schema y usuarios, y aplica
Liquibase
```

La salida debe terminar con «Liquibase command 'update' was executed successfully». Para ver qué haría Liquibase sin aplicar nada:

```
docker compose run --rm liquibase
--defaultsFile=/workspace/liquibase.properties `
  --searchPath=/workspace update-sql
```

- **Contexto de datos:** liquibase.properties usa contexts=dev, que carga cinco medidores de prueba (SW-ESP32-000001 a 000005). En un servidor real se debe ejecutar con --contexts=prod para no cargarlos.

- **Contraseñas de SQL Server:** al menos 8 caracteres con mayúsculas, minúsculas, números y símbolos, sin contener el nombre del usuario; evitar $, comillas simples y dobles. MSSQL_SA_PASSWORD es la misma en todos los repositorios.
- **Nunca editar un changeset aplicado:** Liquibase valida sumas de verificación. Todo cambio va en una nueva carpeta 06-releases/vX.Y con su etiqueta y su rollback.

  Para comprobar la etiqueta aplicada en un schema:

```
SELECT tag FROM devices.DATABASECHANGELOG WHERE tag IS NOT NULL;
```

5. Infraestructura de apoyo

5.1 Redis y Mailpit

Redis guarda la lista de tokens revocados y Mailpit recibe los correos en desarrollo. Ambos se definen en el compose de ms-iam:

```
cd C:\...\sy-water-backend\ms-iam
docker compose up -d               # sy_water_redis (6379) y
sy_water_mailpit (1025 / 8025)
```

Los correos de prueba se ven en http://localhost:8025. Para enviar correos reales se usa Gmail con contraseña de aplicación (smtp.gmail.com, puerto 587, STARTTLS, verificación en dos pasos activa) o Brevo (smtp-relay.brevo.com, puerto 587).

5.2 Mosquitto y RabbitMQ

Ambos se definen en el compose de ms-device. Primero se crea el archivo .env con la contraseña de RabbitMQ y el archivo de contraseñas de Mosquitto con sus dos usuarios: ms-devices (el servicio, solo lee lecturas y escribe órdenes) y sywater-device (compartido por todos los medidores).

```
cd C:\...\sy-water-backend\ms-device
Copy-Item .env.example .env        #
RABBITMQ_PASSWORD=<CONTRASEÑA_RABBITMQ>
docker compose run --rm mosquitto mosquitto_passwd -c -b
/mosquitto/config/passwd ms-devices "<CONTRASEÑA_MQTT_SERVICIO>"
docker compose run --rm mosquitto mosquitto_passwd -b
/mosquitto/config/passwd sywater-device "<CONTRASEÑA_MQTT_MEDIDORES>"
docker compose up -d
docker logs sy_water_mosquitto     # debe decir "mosquitto version 2.0.22
running"
```

Mosquitto solo lee el archivo de contraseñas al arrancar; si se cambia, ejecutar docker compose restart mosquitto. El panel de RabbitMQ queda en http://localhost:15672 con el usuario sywater. Los exchanges y las colas los crean los propios servicios al arrancar.

6. Llaves y clave interna

6.1 Llaves RS256 de los tokens

ms-iam firma los tokens con una llave privada RSA de 2048 bits en formato PKCS#8; los demás servicios los validan con la llave pública.

```
cd C:\...\sy-water-backend\ms-iam
$openssl = "C:\Program Files\Git\usr\bin\openssl.exe"
& $openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out
keys/private.pem
& $openssl pkey -in keys/private.pem -pubout -out keys/public.pem
```

Copiar la llave pública a cada servicio .NET con el nombre iam-public.pem y actualizar su fecha para que se copie a la carpeta de compilación (repetir para places, device, consumption, valve y notification):

```
Copy-Item ..\ms-iam\keys\public.pem ..\ms-places\src\SyWater.Places.Api\
keys\iam-public.pem -Force
(Get-Item ..\ms-places\src\SyWater.Places.Api\keys\iam-
public.pem).LastWriteTime = Get-Date
```

La llave privada nunca sale de ms-iam ni se sube a Git. Si se regenera, hay que volver a copiar la pública en todos los servicios.

6.2 Clave interna entre servicios

Las llamadas internas (por ejemplo, ms-valve a ms-device) usan la cabecera X-Internal-Key. La misma clave, de al menos 24 caracteres, se configura en ms-iam, ms-places, ms-device, ms-valve y ms-notification. Para generarla:

```
$key =
[Convert]::ToBase64String([Security.Cryptography.RandomNumberGenerator]::Ge
tBytes(32)); $key
```

7. Configurar y arrancar ms-iam

ms-iam lee sus secretos de variables de entorno. En PowerShell se definen en la misma ventana donde se ejecuta el servicio (o en la configuración de ejecución de IntelliJ):

Tabla 6

Variables de entorno de ms-iam

Variable	Valor	Obligatoria
SECURITY_APP_PASSWORD	Contraseña del usuario security_app (la del .env de ms-iam-db)	Sí
INTERNAL_API_KEY	Clave interna (sección 6.2)	Sí, para eliminar cuentas, métricas y correo de alertas
DEVICES_URL / PLACES_URL	http://localhost:3003 / http://localhost:3002 (valores por defecto)	No
SPRING_MAIL_HOST, SPRING_MAIL_PORT, SPRING_MAIL_USERNAME, SPRING_MAIL_PASSWORD	Servidor SMTP real; sin ellas usa Mailpit (localhost:1025)	Solo para correo real

```
cd C:\...\sy-water-backend\ms-iam
$env:SECURITY_APP_PASSWORD = "<CONTRASEÑA_SECURITY_APP>"
$env:INTERNAL_API_KEY      = "<CLAVE_INTERNA>"
.\mvnw.cmd test                    # pruebas
.\mvnw.cmd spring-boot:run         # http://localhost:3001/actuator/health →
{"status":"UP"}
```

8. Configurar y arrancar los servicios .NET

Los servicios .NET guardan sus secretos con user-secrets (fuera del repositorio). Cada valor se configura con:

```
dotnet user-secrets set "<clave>" "<valor>" --project
src/SyWater.<Servicio>.Api
```

Tabla 7

Secretos de cada servicio .NET

Servicio	Claves de user-secrets
ms-places	ConnectionStrings:Places, Internal:ApiKey, Cors:AllowedOrigins:0
ms-device	ConnectionStrings:Devices, Mqtt:Password (de ms-devices), RabbitMq:Password,

Servicio	Claves de user-secrets
	Internal:ApiKey, Cors:AllowedOrigins:0
ms-consumption	ConnectionStrings:Consumption, RabbitMq:Password, Cors:AllowedOrigins:0
ms-valve	ConnectionStrings:Valve, RabbitMq:Password, Internal:ApiKey, Cors:AllowedOrigins:0
ms-notification	ConnectionStrings:Notification, RabbitMq:Password, Internal:ApiKey, Cors:AllowedOrigins:0; para correo: Smtp:Host, Smtp:Port, Smtp:Security, Smtp:Username, Smtp:Password, Smtp:FromAddress; opcional Push:ExpoAccessToken

La cadena de conexión sigue este formato, con el usuario de la aplicación de cada schema:

```
Server=localhost,1433;Database=sy-water-db;User
Id=places_app;Password=<CONTRASEÑA>;TrustServerCertificate=True
```

Cors:AllowedOrigins:0 recibe el origen de la web (por ejemplo, http://localhost:8081 o http://<IP_SERVIDOR>:8080); se agregan más orígenes con :1, :2. Las URL entre servicios (Services:*BaseUrl) ya traen localhost con los puertos de la tabla 4 en appsettings.json.

Después de configurar los secretos y copiar la llave pública (sección 6.1), cada servicio se prueba y se arranca desde su carpeta, en este orden: ms-places, ms-device, ms-consumption, ms-valve y ms-notification.

```
cd C:\...\sy-water-backend\ms-places
dotnet test
dotnet run --project src/SyWater.Places.Api       #
http://localhost:3002/health → Healthy
```

Tabla 8

Mensajes esperados al arrancar

Servicio	Mensaje en la consola
ms-device	Now listening on: http://0.0.0.0:3003; conexión a MQTT localhost:1883 y a RabbitMQ
ms-consumption	Consumiendo la cola consumption.readings
ms-valve	Consumiendo la cola valve.device-events
ms-notification	Consumiendo la cola notification.device-events

Si Redis está apagado, los servicios responden 401 a todas las peticiones (fallan de forma segura). Si falta RabbitMq:Host, ms-device arranca pero no publica eventos y los consumidores no reciben lecturas.

9. Firewall

Para que los celulares y los medidores lleguen al servidor, se abren los puertos en PowerShell como administrador (una vez):

```
foreach ($p in 3001,3002,3003,3004,3005,3006,8080) {
  New-NetFirewallRule -DisplayName "Cuida Tu Agua $p" -Direction Inbound -
Protocol TCP -LocalPort $p -Action Allow -Profile Private
}
New-NetFirewallRule -DisplayName "Mosquitto 1883" -Direction Inbound -
Protocol TCP -LocalPort 1883 -Action Allow -Profile Any
```

Mosquitto usa el perfil Any porque Windows suele marcar las redes nuevas como públicas. Las redes de universidades o empresas suelen aislar los dispositivos entre sí; en ese caso se recomienda un punto de acceso propio de 2,4 GHz.

10. Asignar el rol de administrador

El panel no permite convertir a nadie en administrador. El rol se asigna por base de datos a una cuenta ya registrada y verificada:

```
cd C:\...\sy-water-db\ms-iam-db
Get-Content scripts\grant-admin.sql | docker exec -i sy_water_db_dev
/opt/mssql-tools18/bin/sqlcmd `
  -S localhost -U sa -P "<CONTRASEÑA_SA>" -C -b -d sy-water-db -v
ADMIN_EMAIL="admin@ejemplo.com"
```

La persona debe cerrar sesión y volver a entrar, porque los roles viajan dentro del token. Para quitar el rol se usa scripts\revoke-admin.sql de la misma forma.

11. Aplicación móvil y web

11.1 Configuración

En la carpeta save-your-water se copia .env.example como .env y se escriben las URL de los servicios con la IP del servidor en la red local (nunca localhost cuando se prueba desde un celular):

Tabla 9

Variables de entorno de la aplicación

Variable	Servicio y ejemplo
EXPO_PUBLIC_API_URL	ms-iam — http://<IP_SERVIDOR>:3001
EXPO_PUBLIC_PLACES_API_URL	ms-places — http://<IP_SERVIDOR>:3002
EXPO_PUBLIC_DEVICES_API_URL	ms-device — http://<IP_SERVIDOR>:3003
EXPO_PUBLIC_CONSUMPTION_API_URL	ms-consumption — http://<IP_SERVIDOR>:3004
EXPO_PUBLIC_VALVE_API_URL	ms-valve — http://<IP_SERVIDOR>:3005
EXPO_PUBLIC_NOTIFICATIONS_API_URL	ms-notification — http://<IP_SERVIDOR>:3006
EXPO_PUBLIC_RECOMMENDATIONS_API_URL	Servicio de consejos de ahorro (ver nota)
EXPO_PUBLIC_EAS_PROJECT_ID	Solo para notificaciones push (se obtiene con eas init)

Nota: la aplicación espera el servicio de consejos ms-recommendations en el puerto 3007, pero en el repositorio sy-water-backend revisado los consejos están en ms-places (/api/tips). El equipo debe unificar esta configuración antes de publicar.

```
cd C:\...\sy-water-frontend\save-your-water
npm ci                              # instala las versiones exactas del
package-lock.json
npx tsc --noEmit                    # sin errores de tipos
npm test
```

11.2 Aplicación móvil

```
npx expo start -c                  # -c limpia la caché: Expo solo lee .env
al arrancar
```

Para pruebas se abre con Expo Go escaneando el código QR. Las notificaciones push en Android requieren una compilación de desarrollo con EAS (npm i -g eas-cli; eas login; eas init; npx expo install expo-dev-client; eas build --profile development --platform android) y las credenciales de Firebase cargadas con eas credentials; el archivo JSON de la cuenta de servicio nunca va al repositorio.

11.3 Aplicación web y panel de administración

La web de usuario y el panel de administración son la misma aplicación; el panel aparece solo para cuentas con rol ADMIN. Para publicarla se genera la versión estática y se sirve con redirección de todas las rutas a index.html:

```
npx expo export -p web             # genera la carpeta dist/
npx serve dist --single -l 8080     # opción simple
```

Con nginx, la regla equivalente es try_files $uri /index.html; dentro del bloque location /. El origen publicado (por ejemplo, http://<IP_SERVIDOR>:8080) se agrega a Cors:AllowedOrigins de cada servicio .NET y a iam.cors-allowed-origins de ms-iam.

12. Medidores

12.1 Alta de fábrica

Cada medidor necesita un serial (SW-ESP32-NNNNNN), un token secreto y un código de emparejamiento de 8 caracteres. Se generan desde el panel de administración (Medidores → Alta de fábrica), que entrega la hoja de credenciales en CSV, un archivo secrets-<serial>.py por medidor y las etiquetas con QR; el token solo se muestra esa vez. Como alternativa de emergencia existe el script scripts\new-device.ps1 del repositorio ms-device-db (PowerShell 7). La base de datos solo guarda los hash.

12.2 Cargar el firmware

1. Instalar MicroPython en el ESP32: en Thonny, Herramientas → Opciones → Intérprete → Instalar o actualizar MicroPython → Espressif ESP32 / WROOM.
2. Instalar la librería MQTT: Herramientas → Administrar paquetes → micropython-umqtt.simple (o mpremote mip install umqtt.simple).
3. Renombrar el archivo descargado secrets-<serial>.py a secrets.py y completar MQTT_HOST (IP del servidor en la red local, nunca localhost) y MQTT_PASSWORD (la del usuario sywater-device).
4. Copiar al ESP32 todos los archivos .py de la carpeta firmware y secrets.py, dejando main.py al final.

5. Reiniciar con el botón EN o con Ctrl+D. Durante los primeros 3 segundos se puede detener con Ctrl+C.

```
pip install mpremote
cd C:\...\sy-water-hardware\firmware
mpremote cp *.py :                  # incluye secrets.py
mpremote reset
```

Mientras se desarrolla con Thonny conviene poner USE_WATCHDOG = False en config.py, porque el watchdog reinicia la placa si se detiene más de 60 segundos. Antes de instalar en la casa, se vuelve a True.

12.3 Prueba en banco

1. Encender el medidor: el panel de administración lo muestra como «Nunca ha reportado».
2. Conectarse con un celular a la red CuidaTuAgua-XXXX (últimos 4 dígitos del serial); la clave es el código de emparejamiento sin guion.
3. Abrir http://192.168.4.1, elegir la red WiFi de 2,4 GHz y guardar.
4. En pocos segundos el LED queda encendido y el panel muestra el medidor «Conectado» con su versión de firmware.
5. Sostener el botón BOOT 5 segundos para olvidar la red antes de empacar, y pegar la etiqueta en la caja.

13. Verificación de la instalación

Tabla 10

Lista de verificación

#	Prueba	Resultado esperado
1	Abrir /health de cada servicio .NET y /actuator/health de ms-iam	Healthy / UP en los seis
2	Registrar una cuenta desde la aplicación	El código de verificación llega al correo (o a Mailpit)
3	Iniciar sesión y crear un lugar	El lugar aparece seleccionado en Mis lugares
4	Vincular un medidor encendido	Estado «Conectado» y «hace menos de 1 min»
5	Abrir el panel del lugar con agua corriendo	El consumo y el caudal cambian en menos de un minuto

#	Prueba	Resultado esperado
6	Cerrar el agua con el código del correo	El medidor confirma, la válvula queda cerrada y llega la alerta crítica
7	Apagar el medidor más de 10 minutos	Una sola alerta «El medidor se desconectó»
8	Entrar a la web con una cuenta ADMIN	Aparece el panel de administración con el resumen
9	Revisar la cola sywater.dead-letters en RabbitMQ	Sin mensajes

14. Solución de problemas

Tabla 11

Problemas frecuentes

Síntoma	Causa	Solución
Todas las peticiones responden 401	Redis apagado	docker start sy_water_redis
401 solo en un servicio .NET	Llave pública desactualizada en la carpeta bin	Copiar de nuevo iam-public.pem y actualizar su fecha
Login failed for user 'security_app'	SECURITY_APP_PASSWORD distinta a la del .env	Usar la misma contraseña
Missing connection string	Falta el user-secret	dotnet user-secrets list --project ... y configurarlo
Missing 'Internal:ApiKey' o «refused the internal key»	Clave ausente, corta o distinta entre servicios	Usar la misma clave de al menos 24 caracteres en todos
Validation Failed: check sums (Liquibase)	Se editó un changeset ya aplicado	Revertir el cambio y crear una release nueva
THROW 50002 places.places does not exist	Se aplicó ms-device-db antes que ms-places-db	Respetar el orden de la tabla 5
ms-device: MQTT NotAuthorized	Contraseña de ms-devices distinta en passwd y en user-secrets	Recrear con mosquitto_passwd o corregir el secreto
Vincular siempre responde «el medidor no se ha conectado»	El medidor no llega al broker (MQTT_HOST, firewall o WiFi)	Revisar MQTT_HOST, el puerto 1883 y la consola del ESP32
MQTT: no answer from … ECONNREFUSED	Mosquitto apagado o IP equivocada	docker ps; ipconfig
WiFi: no connection after 20 s	Red de 5 GHz, clave o nombre incorrectos	Usar una red de 2,4 GHz
El relé hace clic al reiniciar	GPIO5 es un pin de arranque	Usar GPIO26 y cambiar VALVE_PIN
La placa se reinicia a los 60 s al detenerla	Watchdog activo	USE_WATCHDOG = False mientras se desarrolla

Síntoma	Causa	Solución
Network Error en la aplicación	IP del .env distinta o firewall	ipconfig, reglas de firewall y npx expo start -c
No llegan correos en desarrollo	Mailpit apagado	docker start sy_water_mailpit
«Esta cuenta no es administradora»	Sin rol o token anterior al cambio	Asignar el rol y volver a iniciar sesión
Mensajes en sywater.dead-letters	Mensaje mal formado o que falló 5 veces	Revisarlo en el panel de RabbitMQ y republicarlo

15. Archivos que nunca se suben a Git

Antes de cada commit se revisa git status para confirmar que no aparezcan: archivos .env; keys/*.pem (sobre todo private.pem); user-secrets; secrets.py y secrets-*.py; wifi.json y valve.json del medidor; el archivo passwd de Mosquitto; el CSV de credenciales del alta de fábrica; contraseñas de base de datos, de SMTP o de aplicación de Gmail; la clave interna; el JSON de la cuenta de servicio de Firebase, google-services.json y el token de EAS; node_modules y uploads/.

Referencias

Equipo Cuida Tu Agua. (2026a). *Guías de implementación del proyecto Cuida Tu Agua* [Documentos internos del proyecto].

Equipo Cuida Tu Agua. (2026b). *sy-water-backend* [Código fuente de los microservicios, versión del 9 de octubre de 2026].

Equipo Cuida Tu Agua. (2026c, 9 de octubre). *Documento de diseño del software: Cuida Tu Agua* (Versión 1.0) [Documento interno].